

Start Stop

docker container start -i myHadoop

docker stop myHadoop

hdfs getconf -namenodes

http://localhost:9870

File windows location to docker

docker cp C:/Users/ASUS/Desktop/abul.csv myHadoop:\test

HIVE

Internal (Managed) Tables:

Data Management: Hive fully manages both the metadata (schema, table properties) and the actual data files.

Data Location: Data for internal tables is stored within Hive's warehouse directory (typically /user/hive/warehouse in HDFS by default).

Dropping the Table: When an internal table is dropped using DROP TABLE, Hive deletes both the table's metadata from the Metastore and the corresponding data files from HDFS.

External Tables:

Data Management: Hive only manages the metadata of an external table. The data files reside in a user-specified location in HDFS or another compatible storage system, and Hive does not assume ownership of these files.

Data Location: The location of the data files is explicitly defined during table creation using the LOCATION clause.

Dropping the Table: When an external table is dropped, Hive only deletes the table's metadata from the Metastore. The underlying data files in the specified location remain untouched and are not deleted by Hive.

TeacherInfo

Teacher: Stores teacher details

Teacher_Id,
Teacher_Name,
Email,
Dept_Id.

Department: Stores department details

Dept_Id,
Dept_Name,
Building.

Course: Stores course details linked to teacher and department

Course_id,
Course_name,
Credit,
Dept_id,
Teacher_Id.

Folder Structure

```
hdfs dfs -mkdir -p /TeacherDetail/department  
hdfs dfs -mkdir -p /TeacherDetail/teacher  
hdfs dfs -mkdir -p /TeacherDetail/course
```

```
CREATE DATABASE IF NOT EXISTS TeacherInfo;
```

```
USE TeacherInfo;
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS TeacherInfo.Department (
```

```
    Dept_Id INT,
```

```
    Dept_Name STRING,
```

```
    Building STRING
```

```
)
```

```
COMMENT 'Dept_Details'
```

```
ROW FORMAT DELIMITED
```

```
FIELDS TERMINATED BY ','
```

```
STORED AS TEXTFILE
```

```
LOCATION '/TeacherDetail/Department';
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS Teacher (  
    Teacher_Id INT,  
    Teacher_Name STRING,  
    Email STRING,  
    Dept_Id INT  
)  
COMMENT 'Teacher_Details'  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE  
LOCATION '/TeacherDetail/teacher';
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS Course (  
    course_id INT,  
    course_name STRING,  
    credit INT,  
    dept_id INT,  
    teacher_id INT  
)  
COMMENT 'Course information'  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE  
LOCATION '/TeacherDetail/course';
```

Sample Data for Department Table

1,Computer Science,Building A
2,Mathematics,Building B
3,Physics,Building C

Sample Data for Teacher Table

111,Dr. Asif,asif@ru.ac.bd,1
222,Dr. Humayun,humayun@ru.ac.bd,2
333,Dr. Rokon,rokon@ru.ac.bd,3

Sample Data for Course Table

201,Data Structures,3,1,111
202,Linear Algebra,4,2,222
203,Quantum Mechanics,3,3,333


```
INSERT INTO TeacherInfo.Teacher values(111,Dr. Asif,asif@ru.ac.bd,1);
```

From Command Prompt

```
docker cp C:/Users/ASUS/Desktop/dept.csv myHadoop:\test
```

```
LOAD DATA LOCAL INPATH '/test/Course_Data.csv' INTO TABLE TeacherInfo.Course;
```

Create Relation with Partition Tables

Database Name: TeacherInfo

Teacher_New(Teacher_Id, Teacher_Name, Email, Dept_Id)
LOCATION '/TeacherDetail/Teacher_New';
Partition on Dept_Id.

Department_New (Dept_Id, Dept_Name, Building_Id)
LOCATION '/TeacherDetail/Department_New';
Partition on Dept_Name.

Course_New (Course_Id, Course_Name, Credit, Dept_Id, Teacher_Id)
LOCATION '/TeacherDetail/Course_New';

```
USE TeacherInfo;
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS teacher_new(  
    Teacher_Id INT,  
    Teacher_Name STRING,  
    Email STRING  
)  
PARTITIONED BY (Dept_Id STRING)  
STORED AS PARQUET  
LOCATION '/TeacherDetail/Teacher_New';
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS department_new(  
    Dept_Id    STRING,  
    Building_Id STRING  
)  
PARTITIONED BY (Dept_Name STRING)  
STORED AS PARQUET  
LOCATION '/TeacherDetail/Department_New';
```

```
DELETE FROM department_tmp WHERE  
Dept_Id = '1';
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS course_new(  
  Course_Id  STRING,  
  Course_Name STRING,  
  Credit    INT,  
  Dept_Id   STRING,  
  Teacher_Id INT  
)  
STORED AS PARQUET  
LOCATION '/TeacherDetail/Course_New';
```

What is PARQUET?

Parquet is a columnar storage file format. It is one of the most commonly used formats in big data systems like Hive, Spark, and Impala.

Feature	Description
Columnar format	Stores data by column instead of by row, which is faster for analytics
Efficient I/O	Reads only required columns during queries → faster queries
Compression	Columns with similar data compress better → smaller files
Splittable	Works well with distributed processing (Hadoop, Spark)
Schema support	Self-describing: stores schema with the data
Interoperable	Used by Hive, Spark, Impala, Presto, etc.

id	name	age
1	Ali	25
2	Ana	22

- **Text format** stores:

1, Ali, 25\n 2, Ana, 22

- **Parquet format** stores:

Column-wise:

id: [1, 2]

name: ["Ali", "Ana"]

age: [25, 22]

(plus metadata & compression)

When to Use STORED AS PARQUET

- Queries often access only a few columns
- Dealing with large datasets
- For better performance and storage savings
- To use tools like Spark, Hive, Impala, Presto

Teacher_New(Teacher_Id, Teacher_Name, Email, Dept_Id)

Department_New (Dept_Id, Dept_Name, Building_Id)

Course_New (Course_Id, Course_Name, Credit, Dept_Id, Teacher_Id)

```
LOAD DATA INPATH '/user/hive/warehouse/file.csv' INTO TABLE  
your_partitioned_table;
```

```
docker cp  
D:\\CSE_Dept\\Academic\\CSE_Class\\DistDatabase\\Hadoop\\Dataset\\Course_New.  
csv myHadoop:\\test
```

```
docker cp  
D:\\CSE_Dept\\Academic\\CSE_Class\\DistDatabase\\Hadoop\\Dataset\\Department_  
New.csv myHadoop:\\test
```

```
docker cp  
D:\\CSE_Dept\\Academic\\CSE_Class\\DistDatabase\\Hadoop\\Dataset\\Teacher_New  
.csv myHadoop:\\test
```

```
LOAD DATA INPATH '/path/to/hdfs/datafile.csv' INTO  
TABLE your_partitioned_table PARTITION  
(partition_column=' Dept_Name ');
```

```
LOAD DATA LOCAL INPATH '/test/Course_New.csv' INTO TABLE  
TeacherInfo.Course_New;
```

```
LOAD DATA LOCAL INPATH '/test/Teacher_New.csv' INTO TABLE  
TeacherInfo.teacher_tmp;
```

```
LOAD DATA LOCAL INPATH '/test/Department_New.csv' INTO TABLE  
teacherinfo.department_tmp;
```

```
1) CREATE TABLE zipcodes_tmp(State string,RecordNumber int,Country string,City
   string,Zipcode int) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';

2) LOAD DATA INPATH '/data/zipcodes.csv' INTO TABLE zipcodes_tmp;

3) INSERT OVERWRITE TABLE zipcodes PARTITION(state) SELECT
   RecordNumber,Country,City,Zipcode,State from zipcodes_tmp;
```

```
set hive.exec.dynamic.partition=true;
set hive.exec.dynamic.partition.mode=nonstrict;
```

```
SELECT * FROM teachner_new
WHERE Dept_Id='DPT001'
OR Dept_Id='DPT020' ;
```

```
1) CREATE TABLE zipcodes_tmp(State string,RecordNumber int,Country string,City
   string,Zipcode int) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';

2) LOAD DATA INPATH '/data/zipcodes.csv' INTO TABLE zipcodes_tmp;

3) INSERT OVERWRITE TABLE zipcodes PARTITION(state) SELECT
   RecordNumber,Country,City,Zipcode,State from zipcodes_tmp;
```

```
INSERT OVERWRITE TABLE teacher_new
PARTITION(Dept_Id) SELECT
Teacher_Id,Teacher_Name
,Email,Dept_Id from teacher_tmp;
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS teacher_tmp (
  Teacher_Id  INT,
  Teacher_Name STRING,
  Email       STRING,
  Dept_Id     STRING
)
COMMENT 'teacher_tmp'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
LOCATION '/TeacherDetail/Teacher_Tmp';
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS department_tmp (  
    Dept_Id    STRING,  
    Building_Id STRING,  
    Department_Name STRING  
)  
COMMENT 'department_tmp'  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LOCATION '/TeacherDetail/Department';
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS course_tmp(  
    Course_Id  STRING,  
    Course_Name STRING,  
    Credit     INT,  
    Dept_Id    STRING,  
    Teacher_Id INT  
)  
COMMENT 'course_tmp'  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LOCATION '/TeacherDetail/Department';
```

Database Name: education_db

Purpose: Manage department and course records.

Tables:

1. student_results (*Partitioned*)

- ☐ student_id **INT**
- ☐ subject_code **STRING**
- ☐ marks **INT**
- ☐ grade **STRING**
- ☐ **Partitioned by:** exam_year **INT**, exam_session **STRING**

2. students

- ☐ student_id **INT**
- ☐ student_name **STRING**
- ☐ dob **STRING**
- ☐ department **STRING**

3. courses

- ☐ course_id **INT**
- ☐ course_name **STRING**
- ☐ credits **INT**
- ☐ department **STRING**

Q1 — Database & Table Creation

Write HiveQL statements to create the **student_results** table in education_db, partitioned by (exam_year, exam_session).

1.a) CREATE DATABASE IF NOT EXISTS education_db;

1.b) CREATE TABLE IF NOT EXISTS education_db.student_results (
 student_id INT,
 subject_code STRING,
 marks INT,
 grade STRING
)
PARTITIONED BY (
 exam_year INT,
 exam_session STRING
)
STORED AS PARQUET
LOCATION '/TeacherDetail/Result';

Q2 — Bulk Data Insertion into Partitioned table

Insert data into **student_results** from **result.csv** via temp table called **result_tmp**. You should delete the temp table after data insertion.

```
2.a) CREATE TABLE IF NOT EXISTS education_db.result_tmp (  
    student_id INT,  
    subject_code STRING,  
    marks INT,  
    grade STRING,  
    exam_year INT,  
    exam_session STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LOCATION '/TeacherDetail/Result_tmp';
```

```
2.b) LOAD DATA LOCAL INPATH '/test/student_results.csv' INTO TABLE  
    education_db.result_tmp;
```

```
2.c)  
INSERT OVERWRITE TABLE student_results PARTITION BY(exam_year, exam_session)  
SELECT student_id, subject_code , marks , grade, exam_year, exam_session from result_tmp;
```

```
2.d) Drop table education_db.result_tmp;
```

Q3 — Add a new Partition

Write Hive commands to **add** a partition (exam_year=2025, exam_session='Spring') to the **student_results** table.

3.a) ALTER TABLE **education_db.student_results**
ADD PARTITION (exam_year=2022, exam_session='Fall')
LOCATION '/TeacherDetail/Result';

3.b) SHOW PARTITIONS **education_db.student_results**;

Q4 — Data Insertion into Specific Partition

Insert 3 new records for the above partition. After that verify the insertion.

```
4.a) INSERT INTO TABLE education_db.student_results
PARTITION (exam_year=2025, exam_session='Fall')
VALUES
    (33,'C028',76,'B'),
    (8,'C026',82,'B'),
    (8,'C021',37,'F'),
    (30,'C022',35,'F');
```

```
4.b) SELECT *
FROM education_db.student_results
WHERE exam_year=2024
AND exam_session='Spring';
```

Q5 — Drop Partition

Write Hive commands to **drop** the partition (exam_year=2022, exam_session='Fall') from the **student_results** table.

- 5.a)** ALTER TABLE **education_db.student_results**
DROP PARTITION (exam_year=2022, exam_session='Fall');
- 5.b)** SHOW PARTITIONS **education_db.student_results**;
- 5.c)** MSCK REPAIR TABLE **education_db.student_results**;

Q6 — Query with Partition Filtering

Write a Hive query to display records of students who appeared in the **exam_year=2023** and **exam_session='Fall'**.

```
SELECT
    sr.student_id,
    s.student_name,
    s.dob,
    s.department,
    sr.subject_code,
    sr.marks,
    sr.grade,
    sr.exam_year,
    sr.exam_session
FROM student_results sr
JOIN students s
ON sr.student_id=s.student_id
WHERE
    sr.exam_year=2023 AND sr.exam_session='Fall';
```

Q7 — Join Query

Write a Hive query to display **student_name**, **course_name**, **marks** for all students in the **Computer Science** department.

```
SELECT
    s.student_name,
    c.course_name,
    sr.marks
FROM student_results sr
JOIN students s
ON sr.student_id = s.student_id
JOIN courses c
ON sr.subject_code = CAST(c.course_id AS STRING)
WHERE s.department = 'Computer Science';
```

Q8 — Top Scorer Query

Write a Hive query to find the **student(s) with the highest marks** in the **Spring 2024** session.

```
-- First find the maximum marks in Spring 2024
WITH max_marks AS (
    SELECT MAX(marks) AS max_marks
    FROM student_results
    WHERE exam_year = 2024
    AND exam_session = 'Spring'
)

-- Then join with students & student_results
SELECT s.student_id,
       s.student_name,
       sr.subject_code,
       sr.marks,
       sr.exam_year,
       sr.exam_session
FROM student_results sr
JOIN students s
    ON sr.student_id = s.student_id
JOIN max_marks m
    ON sr.marks = m.max_marks
WHERE sr.exam_year = 2024
    AND sr.exam_session = 'Spring';
```


Q9 — Aggregation

Find the **average marks** per department for the exam_year **2024**.

```
SELECT
    s.department,
    AVG(sr.marks) AS avg_marks
FROM student_results sr
JOIN students s
ON sr.student_id = s.student_id
WHERE sr.exam_year = 2024
GROUP BY s.department;
```

Q10 — Top Scorer Query

Write a Hive query to find the **student(s) with the highest marks** in the **Spring 2024** session.

Find the maximum marks in Spring 2024

```
WITH max_marks_cte AS (  
    SELECT MAX(marks) AS max_marks  
    FROM student_results  
    WHERE exam_year = 2024 AND exam_session = 'Spring'  
)
```

Select student(s) who scored those marks

```
SELECT  
    sr.student_id,  
    s.student_name,  
    sr.subject_code,  
    sr.marks  
FROM student_results sr  
JOIN max_marks_cte mm  
    ON sr.marks = mm.max_marks  
JOIN students s  
    ON sr.student_id = s.student_id  
WHERE sr.exam_year = 2024 AND sr.exam_session = 'Spring';
```